SpectraSource Architecture Document

1 Overview

SpectraSource turns one source (text, PDF, DOCX, image, audio or video) into every deliverable the operator selects from seven output types, shaped by six generation parameters. The backend extracts the source to text, generates each deliverable through a swappable LLM provider, validates it against a strict schema and offers it as a ready-to-use file. Every job and artefact is written to a tamper-evident, hash-chained audit log.

2 System architecture

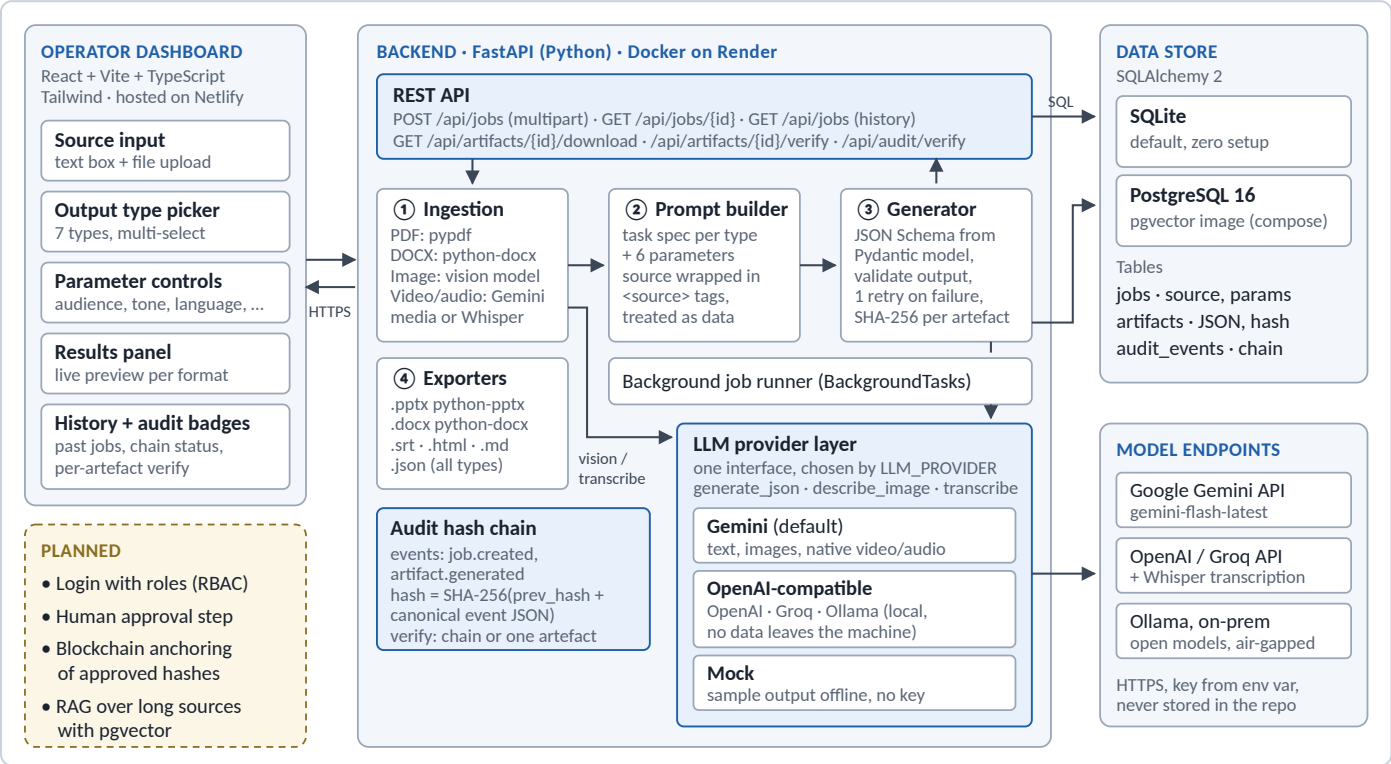


Figure 1. Components and data flow. Everything outside the dashed **PLANNED** block is built and running on the repository's `main` branch; blue boxes hold the core logic.

3 Request lifecycle

1. **Submit.** The dashboard sends `POST /api/jobs` as multipart form data: output types, parameters, and text and/or one file (25 MB cap).
2. **Ingest.** Within the request, the file becomes plain text: PDF and DOCX are parsed locally, images are read by a vision model, and video or audio goes to Gemini's media API (or Whisper) for a transcript with timestamped visual notes. Typed text is added as context; the total is capped at 60,000 characters.
3. **Record.** A job row and one pending artefact per output type are saved, the source's SHA-256 is computed, and a `job.created` event is appended to the audit chain. The API answers `202 Accepted` at once.
4. **Generate.** A background task builds one prompt per output type and calls the provider, which must return JSON matching that type's schema (one retry if invalid). Each artefact succeeds or fails on its own, is hashed and is logged as `artifact.generated`.
5. **Deliver.** The dashboard polls `GET /api/jobs/{id}`, previews each artefact as it finishes, shows whether its hash verifies, and downloads files rendered from the stored JSON on demand.

4 Deployment and operations

Environment	Setup
Hosted demo	Dashboard built and served by Netlify (`netlify.toml`). API on Render as a Docker web service (`render.yaml`) with Gemini selected, the key held in Render's environment and a `/health` check. The free tier sleeps when idle; the dashboard shows a "connecting" badge while it wakes.
Local / on-premise	Uvicorn with SQLite plus `npm run dev`, or `docker compose up` for the API with PostgreSQL. The mock provider needs no key; with Ollama, the same image runs on an internal network with no external calls.
CI	GitHub Actions on every push and pull request: backend pytest on Python 3.10 and 3.12, frontend lint and build.

5 Components

Component	Technology	Responsibility
Dashboard	React 19, Vite, TypeScript, Tailwind CSS	Single-page app: health header, source input, format picker, parameters panel, results panel with a live preview per format, history sidebar, and audit badges.
REST API	FastAPI, Pydantic v2, Uvicorn	Validates input, enforces upload and length limits, creates jobs, serves results, paged history, downloads and audit checks. OpenAPI docs at `/docs`.
Ingestion	pypdf, python-docx, provider media calls	Normalises every input kind into one source text so all generators share the same facts.
Generators	Per-type spec and Pydantic schema	One task instruction and one output schema per deliverable; schema-checked JSON means the UI and exporters never parse free text.
LLM providers	httpx clients behind one interface	Gemini (default), any OpenAI-compatible server (OpenAI, Groq, local Ollama) and an offline mock. Switching is a configuration change.
Exporters	python-pptx, python-docx	Turn stored JSON into usable files on request.
Audit chain	SHA-256, canonical JSON	Append-only log; each event's hash covers the previous one, so any edited or deleted row is detected. One artefact can also be re-hashed and checked against its event.
Data store	SQLAlchemy 2; SQLite or PostgreSQL 16	Jobs, artefacts and audit events. Docker Compose runs Postgres from the pgvector image.

6 Output types, parameters and exports

Output type	Structured content	Download as
Video package	Narration script, scene-by-scene storyboard, scene descriptions, subtitles, visual recommendations	.docx · .srt · .md
LinkedIn post	Hook, body, call to action, hashtags	.md
Twitter/X	Single tweet or numbered thread, each tweet ≤ 280 characters	.md
Advisory	Severity, summary, affected systems, description, impact, recommendations	.docx · .md
Infographic	Headline, key message, sections with key stats, icons, layout and palette	.html · .md
Executive summary	Bottom line up front, key points, implications, recommended actions	.docx · .md
Presentation	5–10 slides with bullets, speaker notes and a visual suggestion each	.pptx · .md

All types also export JSON. Each job carries six operator parameters that go into every prompt: **target audience, tone, output language, level of detail** (brief, medium, detailed), **communication objective** and **content style**. Several types share one extracted source, so facts stay consistent, and one failure does not stop the others.

7 Security and trust

BUILT

- **Prompt-injection guard:** source sits inside `<source>` tags and the model is told to treat it as data and ignore instructions in it, including in audio and video.
- **Grounding rule:** only facts from the source; never invented numbers, names, dates or CVEs.
- **Strict output schema:** responses that do not validate are rejected, never shown.
- **Tamper evidence:** SHA-256 of every source and artefact, linked in the audit hash chain.
- **Data sovereignty:** a local Ollama provider keeps all content on the organisation's own hardware.
- **Secrets** only in environment variables; CORS limited to the dashboard.

PLANNED

- **Login with role-based access** (operator, reviewer, admin), with the user recorded as the audit event's actor in place of today's `anonymous`.
- **Human approval step** before an artefact is final.
- **Blockchain anchoring:** write the hashes of approved artefacts to a ledger, so approval records can be checked independently of our database.
- **RAG with pgvector** so long sources are chunked and outputs cite source passages.
- Also: local Whisper, keyframe captions for silent video, a task queue, PDF export.